

Programming Language Design

Module 4: Data types

Slides © Grant Williams, [CC BY-SA 4.0](#).

This is an open educational resource.

Feel free to submit fixes, improvements, and new material [here](#).

Data types

We learned about simple built-in datatypes last lecture. This time, we're going to learn how they work, and how you can make your own.

Obviously, you aren't going to make your own `Int`. Haskell has you covered there. There's also `Int8`, `Int16`, etc.

But, datatypes are a little different in functional languages. There are the equivalent of structs, sure, but also enums. So let's talk about those...

Enums in C

Enums in C let you list every value a type can have.

They are also a way to create constants:

```
typedef enum {  
    FALSE,  
    TRUE  
} MyBool;
```

Here, a MyBool is supposed to have one of two values:

```
MyBool a = TRUE; // fine  
MyBool b = FALSE; // also fine  
MyBool c = 15; // may emit a warning, although technically permitted
```

Enum variants

The items inside the enum are called "variants" in C.

The compiler creates a constant for each variant.

So now FALSE and TRUE are defined.

By default, the first variant gets value 0 and the second gets 1

So you can do this:

- `int x = FALSE; // x is 0`
- `x = TRUE; // now it is 1`

But you can also do this:

`int x = 20;` , which you probably don't want...

Enums give some safety

Suppose we add another enum:

```
typedef enum { RED, GREEN, BLUE } MyColor;
```

The compiler can help you catch errors if you actually use the enum wrongly:

```
MyBool a = FALSE; // okay  
MyBool b = RED; // generates a warning
```

`b` is a `MyBool`, but we're setting it to a `MyColor`

Behind the scenes `RED == 0` and `FALSE == 0`, so if we were using constants, there would be no warning that we were setting the wrong enumeration value.

[Important: does it make sense how we need enums here to detect the error, and how regular constants aren't enough?]

Setting the values of variants

One last thing, just to make sure you've seen it:

You can set the initial value or all the values if you want:

```
typedef enum {  
    FALSE = 0,    //set the initial value  
    TRUE         //this value is now 1 automatically.  
} MyBool;  
  
typedef enum { RED = 20, GREEN = 21, BLUE = 22 } MyColor;
```

It's like setting a bunch of constants but you'll be warned if you use one in the wrong place (like assigning a bool to a color).

Questions?

Why do Enums exist?

They are actually extremely common in actual professional code, but you may not have seen them much in your studies so far.

Enums are very common as a way to describe states or commands.

For example, a webserver has certain `modes` for connections, where for each one it might be parsing requests, authenticating, or sending data.

And games have states, too...

Gamestate enum

Suppose we're making a basic video game in C.

It's common to implement states as enums:

```
typedef enum {  
    STATE_SPLASH_SCREEN = 1, //skip state 0, maybe use it to debug  
    STATE_MAIN_MENU,  
    STATE_CHAR_SELECT,  
    STATE_IN_GAME,  
    STATE_LOADING  
} GameState;
```

How do we use them?

States in old-school C

Typically there's some kind of if-statement or switch where we do something depending on which state we're on:

```
if( current_state == STATE_SPLASH_SCREEN )
    update_splash_screen( time_remaining );
else if( current_state == STATE_MAIN_MENU )
    update_main_menu();
else if( current_state == STATE_CHAR_SELECT )
    update_char_select( secret_chars_unlocked );
else if ...
```

(protip, if you start your states at 0 or close to 0, you can then use the state variable as an index into an array of function pointers.

```
GameState state_impls[] = { &DoDebug, &DoSplash, &DoMenu, ... }; )
```

Explanation

Here, we're using an enum to describe all the states we could be in.

When we're in a state, we do some unique action. I'm glossing over that part.

The only issue is, the data the state needs to access is probably global with this approach. Not the end of the world, but not always good. For example, in the webserver example, there might be thousands of connection objects.

It's possible to attach each state enum to a `void*` data. This is called a "tagged union". But it's not 100% typesafe: we have to make sure to always interpret the `void*` ...

```
typedef Connection {  
    ConnectionState state; // CONNECTING, SENDING, PARSING, etc.  
    void* data; // unique data for each state.  
}
```

Questions?

Back to Haskell

So, what about our game we're writing in Haskell?

Haskell doesn't have `void*`, (technically, it's possible but not recommended).

Instead, it has typesafe ways of doing this.

We want to associate data for each state:

- For the splashscreen, maybe how much time has passed so we can fade in.
- For the main menu, no extra data.
- For character select, which extra characters have been unlocked.
- For the in-game state, probably a lot of data. Which characters were selected, their health and meters, the time left, the stage, etc.
- For the loading state, what data are we loading?

Back to Haskell (2)

At first glance, Haskell seems like it has an exact substitute for enums.

We use `data` to define a new datatype...

```
data GameState =  
    | Splash  
    | MainMenu  
    | CharSelect  
    | Loading  
    | InGame
```

Use the vertical bar `|` to separate variants.

We can then define a variable of this type:

```
startingState :: GameState  
startingState = Splash
```

Differences between C and Haskell enums/datas

You don't define the actual number that each variant takes. (that's hidden by the compiler)

You separate with `|` instead of `,`

We use initial camel case instead of all-caps snake case.

In Haskell you *may not* mix variants from other types.

Suppose `f :: GameState -> Bool`

This is allowed: `f InGame`

This is forbidden: `f Red`

Haskell is very strictly-typed here. The variants are not just constants. They are fully unique values that can't be mixed with incompatible types.

Constructors

In fact, Haskell does not call these *variants*...

It calls them **constructors**!

Yes, the same term as OO, but used differently.

Consider this datatype: `data Rgb = Red | Green | Blue`

Here, `Red`, `Green`, and `Blue` are all constructors in Haskell.

They are ways of constructing data. If you write `Red`, you have constructed an `Rgb`.
Likewise for `Green` or `Blue`.

Knowledge check 1

1. Define a type with two constructors.
2. Define a different type with four constructors.
3. Define a function and its type that takes the first type and returns the second.
4. Call the function. What is the result?

Knowledge check 1 answers

1. `data Blip = blip | blop`

2. `data Foo = foo | bar | baz | quux`

3.

```
blorp :: Blip -> Foo
blorp blip = foo
blorp blop = baz
```

4. `blorp blip`. The result is `foo`.

Questions?

Real data types

Now we get to something that is clasically confusing. We're going to learn about true tagged unions in a functional language.

This feature isn't just in functional languages anymore. Rust has it for example. It's a nice way of implementing datatypes.

You see, these data types don't just have to be distinct constructors. Each constructor can also carry different data.

For example, what if I want a data type to represent single data values, *or* pairs of values?

We can still do that, and this is where Haskell's data types can do something that a basic C enum cannot do...

Single or Double example

```
data SingleOrPair =  
    Single Int  
  | Pair Int Int
```

Here we've created a new datatype with two constructors: one for single ints, and one for pairs of ints.

`SingleOrPair` is a type. `Single` and `Pair` are constructors, not types.

These two constructors require either one `Int` or two in order to be called.

This is weird, so let me show you how this is used.

Single or Double example

Let's say I want to define a Single or pair:

```
s :: SingleOrPair
s = Single 20 -- this is fine

t :: SingleOrPair
t = Pair 30 40 -- also fine
```

A variable of type `SingleOrPair` can be filled with either a `Single` or `Pair`.

Constructors are not types

We can think of `Single` and `Pair` as different families of values. A `SingleOrPair` can either be a `Single` or a `Pair`. However `Single` is not a type, and neither is `Pair`.

You *cannot* do this: `s :: Pair`

because the thing after the `::` is expected to be a type, but `Pair` is a constructor.

Think of `Single` and `Pair` as functions, rather than types, and this will make sense.

But with the `data` keyword, we are expected to exhaustively define all the ways of creating the value.

Datatypes are exhaustive

This is an important point. When you create a data type in Haskell, you must list *all* of its constructors.

This means, Haskell knows at that point, all of the possible values it could take.

But how do we get the values out? Suppose I have a `Pair 20 30` ? How do I get that `20` and `30` back out of it?

Pattern matching

Pattern matching is the main, basic way we get data out of data structures.

Not only in Haskell, but it's fairly common in many functional languages.

One simple way to do it is using case expressions...

Case expressions

This Haskell code will take a `SingleOrPair` and either print the value if it's a single, or print the sum of both values if it's a pair:

```
main = do
  let x = Pair 20 30
  case x of
    Single y -> print y
    Pair y z -> print (y + z)
```

(remember that `let` expressions don't have an `in` clause inside of `do` blocks. We'll talk more about `do` blocks when we get to monads.)

Here, we're using `case` to consider all the possible values that `x` could have. Crucially, *Haskell will warn us if we miss one!* Something that C enums can't always do.

Pattern-matched definitions

Alternatively, we can define functions on particular constructors:

```
singleOrSum :: SingleOrPair -> Int
singleOrSum (Single x) = x
singleOrSum (Pair x y) = x + y
```

This is saying: `singleOrSum` is a function that, when it receives an `Int` named `x`, it just returns `x`, but when it receives a `Pair` of `x` and `y`, it returns `x + y`.

This works exactly the same as pattern matching on basic values like `Int`:

```
fib0 0 = 0
fib0 1 = 1
fib0 n = fib0 (n - 1) + fib0 (n - 2)
```

Here, `Pair 20 30` is just as much a value as `1`.

Knowledge check 2

1. Rewrite that case expression so that there is only one `print`. We're repeating code unnecessarily.
2. Define a data type called `NilOrTriple` which has a constructor `Nil` which takes no arguments, or a triplet of `Ints`.
3. Write a function that returns the second value if it is a `Triple`, or `0` if it is a `Nil`. Include its type.

Knowledge Check 2 answers

1. `print (case x of Single y -> y ; Pair y z -> y + z)`

(note: we can use semicolons instead of newlines)

Make sure this one makes sense: `case` is an expression, not a statement. It has a value that can be substituted for the argument of `print`.

2. `data NilOrTriple = Nil | Triple Int Int Int`

3.

```
secondOrZero :: NothingOrTriple -> Int
secondOrZero x =
    case x of Nil -> 0 ; Triple _ y _ -> y
```

```
-- or, this is nicer imo vvv
secondOrZero Nil = 0
secondOrZero (Triple _ y _) = y
```

Questions?

Behind the scenes

Data types in Haskell work in roughly the same way as they do in C behind the scenes.

There is a number stored in the value for each constructor.

This number is hidden to you, but Haskell keeps track of it.

That means there's a small bit of overhead. And if there is only one constructor, that overhead is optional...

Newtype

Sometimes we know that the data type will have exactly one constructor that takes exactly one piece of data.

This happens when we use types as *wrappers*. That is, we have a basic type, like a `String`, but it represents something specific, like an `Id`.

We could do this:

```
data Id = Id String
```

(you're allowed to have constructors with the same name as the datatype)

Here, we're saying an `Id` is just a string. But the user has to explicitly use the constructor so that it's clear they know the string is being used as an `Id`.

Newtype (2)

However, this might involve overhead. What if the Haskell implementation is storing data about the variant, but there's only one variant so it's pointless?

There's a keyword you can use to suggest to Haskell that it should not do this:

```
newtype Id = Id String
```

Here, `newtype` is just like `data`, but it can *only* be used when there is a single constructor that has exactly one field.

Not zero fields: `newtype Blark = Blorp -- wrong!`

Not two fields: `newtype Bloink = Blip Int Float`

Exactly one field `newtype Blomp = Bloop Int`

Newtype (3)

Newtype is kind of a janky feature. You don't have to use it, it was added as a tiny optimization.

In programming language design, we often don't like features like this. We generally like features that are *orthogonal*.

Orthogonal features are those that behave the same way consistently, or that interact well with a large number of other features.

Orthogonality

For example, functions in Haskell are a very orthogonal feature. Their basic mechanics are responsible for almost all the features of the language.

Case expressions are also orthogonal: case can be used to match any expression.

`newtype` is not very orthogonal. It only works with a very specific kind of datatype. It's the kind of feature that makes programming language designers say "ugh".

So does that mean we shouldn't have it?

Unfortunately we need it

`newtype` actually does fill a tiny but important role.

You might think "just check to see if there is only one constructor with one field and then omit the extra information that `data` stores."

The problem is that the extra information here also allows the constructor to store erroneous values without crashing the program.

This is getting esoteric, but there is a certain value, called "bottom" (written $\perp$) that basically represents an "unreachable" value.

Unfortunately we need it (2)

With lazy evaluation, a constructor that holds a field with bottom in it will not be considered an error by itself. In rare circumstances this can be desirable. For example, deliberately leaving an undefined value when I know I won't need it but I want to crash if I'm wrong.

I don't expect this all to make sense right now. The main point I'm making is that programming languages aren't things that emerge naturally, or that are necessarily perfect.

Instead, they are things that are engineered. And engineering involves tradeoffs. Every language is going to have something janky in it, and Haskell is no exception.

One more option

There's one more, slightly less janky way of defining new datatypes.

The `type` keyword is useful. It lets you define a *type alias*.

A type alias is literally just another name for a type. Like this:

```
type Blip = String
```

Now, everytime I write `Blip`, it will be interpreted as `String`.

```
f :: Blip -> Blip
f x =
  | x == "Hi" = "Hi back at you!"
  | otherwise = "You said: " ++ x
```

(reminder that `++` is string concatenation)

Knowledge check 3

1. Use `newtype`. Just use it to define some kind of type that takes a `String` as a field.
2. Now use `type` to define an alias of that type.
3. Now redefine the function `f` on the previous slide to take your alias. Be sure to use the constructor when defining the function. You can't create an instance of a custom type defined with `data` or `newtype` without calling the constructor, and you can't get data out without some kind of pattern matching (either directly or in a function).

Knowledge check 3 answers

1. `newtype Blop = Blop String`

2. `type Bloop = Blop`

3.

```
f :: Bloop -> Bloop
f (Blop "Hi") = "Hi back at you!"
f (Blop whatever) = "You said" ++ whatever
```

Questions?

Let's make a list!

Now that we understand datatypes, let's build a linked list!

Ignore the fact that we already have them. We're trying to understand them better.

When you took data structures, you probably saw a linked list node like this (Java):

```
class Node<T> {  
    Node<T> next;  
    T data;  
}
```

So it seems like we could do something like this...

Wrong list

```
data Node t = Cons t (Node t)
```

(note: a list node in functional languages is often called a `Cons`. It comes from Lisp)

How do we interpret this?

There is a type, `Node t`. This type is *parameterized*. We're saying that the type `Node` is not complete by itself. It requires an additional type.

Just like in Java we would write `Node<Integer>`, in Haskell we write `Node Integer`.

Wrong list (2)

```
data Node t = Cons t (Node t)
```

Once we provide a `t` for the type, we now know the type of the constructor.

For example, the type `Node Int` has a constructor that takes an `Int` (for the first argument) and a `(Node t)` for the second argument.

If you're curious, Haskell also has "records" which are like classes:

```
data Node t = Cons { val :: t, next :: (Node t) }
```

We'll cover these later.

But there's a reason why the title of the last two slides is "Wrong list"...

Can anyone tell me why we cannot construct this type?

No null

In Java, if we want a node to be the last node in the list, we make it `null`.

`null` is regarded by its inventor (computer scientist [Tony Hoare](#)) as a [huge mistake](#).

I agree with him. `null` basically adds a special untyped value to the language that is a member of every type.

For example, you thought you defined a simple color:

```
data Color = Red | Green | Blue
```

But if Haskell had `null`, the color could also be `null`. So there would be 4 possibilities.

No null (2)

Haskell data types are always exhaustive.

If I have a case expression like this:

```
case color of
  Red  -> ...
  Green -> ...
```

The compiler knows there's an issue because we're not handling `Blue`.

We get a warning.

This is actually a great safety feature: it lets us be sure that we have considered every possibility. There's no runtime crash if we handle all the values.

No null (3)

Null would prevent this nice feature. If every type can be null, then every single time we use a value we get from another function, it could potentially be null.

All of these null checks would be annoying, so most languages with pervasive nulls just don't require checking.

So you end up with "null reference exception" sometimes.

This is not a problem with languages with stricter type systems (like Haskell).

Fixing the list

So I said the list was wrong:

```
data Node t = Node t (Node t)
```

What's wrong with it?

There's nothing we can put in the "next" pointer (the `Node t`) when the list is over.

Like, suppose I want a list with the values `20` and `30`:

```
Node 20 (Node 30 (... uh?))
```

What goes in the `... uh?` ? There is no null?

[any ideas on how to fix this?]

Fixing the list (2)

Just add a constructor:

```
data Node t = Nothing | Cons t (Node t)
```

Now, there are two kinds of `Node`. A `Nothing`, and a `Cons`.

If we want an empty list, we just make a `Nothing`:

```
justAnEmptyList :: Node Int -- I gave it a type...  
justAnEmptyList = Nothing -- ...Nothing can be a list of anything
```

And if we want a list of one:

```
aSingletonList = Cons 20 Nothing
```

Knowledge check 4

1. Define a binary search tree node data type that works on Integers.
2. Write a function (and its type) that inserts a node into the tree.
3. Write a function (and its type) that determines how many nodes are in the tree.
4. Homework: write a function and its type that looks up a node in the tree.

Knowledge check 4 answers

1.

```
data Bst = Empty |  
         Node Integer Bst Bst  
         deriving Show
```

2.

```
bstInsert :: Integer -> Bst -> Bst  
bstInsert val Empty = Node val Empty Empty  
bstInsert val (Node x left right)  
  | x == val = (Node val left right)  
  | x < val = (Node x (bstInsert val left) right)  
  | otherwise = (Node x left (bstInsert val right))
```

Knowledge check 4 answers (2)

3.

```
bstSize :: Bst -> Int
bstSize Empty = 0
bstSize (Node _ left right) =
    1 + bstSize left + bstSize right
```

4. Hint: the answer for `bstInsert` gives a clue for how to think about how to build this function recursively.

Try testing your tree:

```
main = do
    let tree = bstInsert 2 (bstInsert 3 (bstInsert 1 (bstInsert 2 Empty)))
    print tree
    print (bstSize tree)
```

Questions

Sum types vs product types

One brief bit of terminology.

Suppose we have these data types:

```
data PrimaryColor = Red | Green | Blue
data ShinyColor = Bronze | Silver | Gold
data Color = Primary PrimaryColor | Shiny ShinyColor
```

1. How many distinct values of `PrimaryColor` are there?
2. How many distinct values of `ShinyColor` are there?
3. How many different values of `Color` are there? List them.

Sum types vs product types (2)

1. 3

2. 3

3. $3 + 3$, which is 6. Primary Red, Primary Green, Primary Blue, Shiny Bronze, Shiny Silver, Shiny Gold.

Notice that the number of `Color`s are the number of `PrimaryColor`s plus the number of `ShinyColor`s. It's the sum.

When we use *alternation* (the `|` symbol) to define a datatype, we are defining something called a *sum type*.

Sum types are types in which the number of values is the sum of all the alternatives.

Sum types

Enums and unions in C are another example of sum types.

If you have an enum with 5 variants, and an enum with 10 variants, then if you create union of both enums, it will have 15 possible (valid) values.

In general, Haskell's data types offer features of both unions and enums. However, they can also be *product types*...

Product types

Let's use `PrimaryColor` and `ShinyColor` again.

How many different values of `(PrimaryColor, ShinyColor)` are there?

This is a tuple of two values, one primary and one shiny.

Product types (2)

There are 9 possibilities, not 6!

```
(Red, Bronze), (Red, Silver), (Red, Gold), (Green, Bronze), (Green, Silver),  
(Green, Gold), (Blue, Bronze), (Blue, Silver), (Blue, Gold)
```

This is also the case if we declare a data type with multiple arguments:

```
data ColorCombo = ColorCombo PrimaryColor ShinyColor
```

Record types (which we'll talk more about later) and structs in C are also product types:

```
struct ColorIntensity {  
    Color c; // RED, GREEN, or BLUE  
    Intensity i; // DIM, NORMAL, BRIGHT  
}
```

Here, `struct ColorIntensity` has 9 valid values.

Questions?

Lists are already built-in

Let's stop using custom Cons. Lists are already in the language.

Who can do pattern matching on lists directly:

```
isEmptyList :: [a] -> Bool
isEmptyList [] = True
isEmptyList _ = False
```

In fact, the operator that creates lists, `:`, is already called the "Cons operator".

```
push7 :: [Int] -> [Int]
push7 list = 7 : list
-- or, more elegantly
push7 = (7:)
```

Using the cons operator

Deconstructing is when we use pattern matching to pull data out of a data type.

In the same way that we can destructure a `Cons` or a `Bst`, we can also destructure a `:` that is used to build a list.

For example, the `head` function returns the first element in a list. Here's how we can write it with pattern matching:

```
head :: [a] -> a
head (x : xs) = x
```

But wait...

There's the possibility of a runtime error here, right after I said Haskell was so cool with its type system.

If the list is empty, it won't match `x : xs`, because it's not a `Cons`.

Sometimes we actually *do* want the ability to say "hey, sometimes we can return nothing".

But we want it to be clear *when* we can return nothing.

We don't want literally every type to have a "null" option, but we want the ability to let some types have 'Nothing' as an option.

Maybe

The type is called `Maybe`. It looks like this:

```
data Maybe a = Nothing | Just a
```

So a `Maybe Int` has two constructors. "Nothing" (which is a constructor for every type of maybe) and "Just Int". That is:

```
x :: Maybe Int
x = Nothing -- valid

y :: Maybe Int
y = Just 20 -- valid

z :: Maybe Int
z = 20 -- invalid. 20 is not one of the constructors: Just or Nothing.
```

Maybe is not the same thing as null

The closest approximation of Maybe in other languages is not null, but rather "nullable".

`null` is a value. It's not a type. `Maybe t` is a type.

Specifically, a parametrized type (an "of" type).

For those who took Typescript with me, `Maybe t` is like `t | undefined` or `t | null`.

If a function returns a Maybe, it's saying "it might be nothing".

How do we make a version of `head` that returns `Nothing` when the list is empty, and `Just x` when the list starts with `x`?

Maybe (2)

```
head' :: [a] -> Maybe a
head' [] = Nothing
head' (x : xs) = Just x
```

Here, if we try to get the head of an empty list, it ends up being "Nothing".

However, if there actually is a list, the result is `Just` the first element.

But once the value is wrapped in a `Maybe`, how do we get it out?

Same as always

You use pattern matching to get values out of data structures:

```
print (  
    case head' myList of  
        Just x -> show x  
        Nothing -> "nothing"  
)
```

But wait, who can tell me why this is a little goofy? Is there a better way to get the value out other than `head'` ?

A better option

What if we just used pattern matching to begin with?

Head functions can be useful (when we compose), but we don't really need one here.

```
print (  
  case myList of  
    (x : xs) -> x  
    _ -> "nothing"  
)
```

Here, we just used pattern matching directly on the list to get the head rather than an additional function to get the data out.

Either

There's one more basic datatype that we should cover: `Either` :

```
data Either a b = Left a | Right b
```

Notice, this data type has two type parameters.

`Either` represents things that can be "either" an `a` , or a `b` .

For example `Either String Int` can store a `String` or an `Int` . If we want it to store a `String` , we use the `Left` constructor, and if we want it to store an `Int` we use the `Right` constructor.

```
aString :: Either String Int; aString = Left "Hello"  
anInt   :: Either String Int; anInt = Right 44
```

Either (2)

But why? Why not make a custom type?

```
data StringOrInt = SoIString String | SoInt Int
```

We can, but by using an `Either`, our type gains some magical abilities. Haskell allows you to extend data types. We'll talk about how later, but `Either` has been heavily extended to make it useful for error handling.

Basically, the `Right` type is the "good" type (because it's "right") and the `Left` type is the "error" type. (note: for those of you who program in Rust, it's just `Result<E, T>`)

So if have a function return `Either`, you are saying "this function can return a `b`, or it can error-out with an `a`". You need to check if the result was a `Left` or a `Right` to know which one happened.

Questions?

Knowledge check 5

1. Write `sum'`, which computes the sum of a list. Include the type.
2. Write `tail'` which returns a maybe, but it returns the rest of the list excluding the head. Include the type.
3. Write `headOfTail` which returns the head of the tail if there is one, or `Nothing` otherwise. Include the type.
4. Rewrite `tail'` from 3 to make it return an `Either` instead of a `Maybe`. The left type should be `String`, and it should return `"the list was empty"` if the list is empty.
5. Rewrite `headOfTail` which returns the head of the tail if there is one, or `"the list was empty"` if the list was empty, or `"the tail was empty"` if there was a tail, but it had no head.

knowledge check 5 answers

```
sum :: [Integer] -> Integer
sum [] = 0
sum (x : xs) = x + sum xs
```

```
tail' :: [a] -> Maybe [a]
tail' [] = Nothing
tail' (x : xs) = Just xs
```

```
headOfTail :: [a] -> Maybe [a]
headOfTail [] = Nothing
headOfTail [_] = Nothing -- or headOfTail (x : []) = Nothing
headOfTail list = Just (head (tail list))
```

knowledge check 5 answers (2)

```
tail'' :: [a] -> Either String [a]
tail'' [] = Left "the list was empty"
tail'' (x : xs) = Right xs
```

```
headOfTail' :: [a] -> Either String [a]
headOfTail' [] = Left "the list was empty"
headOfTail' [_] = Left "the tail was empty"
headOfTail' list = Right (head (tail list))
```

The function application operator, \$

There's a weird operator that you've probably seen if you've been diligent about doing your Haskell codewars practice (and if you haven't, please do this. It's fun and good.)

It's the weird dollar sign:

```
print $ head [1, 2, 3]
```

This has the same meaning as

```
print (head [1, 2, 3])
```

But it has one less keystroke, and it also avoids lots of nested parentheses:

```
print $ head $ tail [1, 2, 3] -- prints 2
```

\$ (2)

How does it work?

One thing we've glossed over but will cover later: operators in Haskell are just functions.

You can actually add more operators if you want. You just tell it the associativity, the precedence, and what you want it to do.

They have types, and behind the scenes, Haskell just maps them to a function.

Let's check the type in `ghci` :

```
ghci> :t ($)
($) :: (a -> b) -> a -> b
```

Can anyone help me interpret this type signature?

\$ (3)

This is a binary operator. That is, an operator with two operands.

The first argument is a function `(a -> b)`. All we know about it is that it takes one argument and returns one, so it's a normal lambda function.

There is no restriction at all on the types of the operands.

The second argument is just an `a`. That is, whatever the type the function takes.

\$ (4)

So all this operator does is take a function from `a -> b`, a value of type `a`, and then it applies the function and returns the result of type `b`.

It's literally just "give me a function, now give me an `a`. I will apply the function to it.

Also, it associates to the right. [What does that mean?]

In general, what is the difference between left, right, and full associativity?

Associativity

An associative operator is one in which it doesn't matter whether you evaluate the left side first or the right side first.

For example, `+` and `*` are both associative operators, because `(1 + 2) + 3` and `1 + (2 + 3)` are the same. Same for times.

`-` and `/` are not associative. `(2 / 3) / 4 != 2 / (3 / 4)`

So if `/` is not associative, what does this mean? `2 / 3 / 4`?

Left vs right associativity

Haskell interprets `2 / 3 / 4` as `(2 / 3) / 4`. This is called left associativity.

Compare to exponentiation, `2 ^ 3 ^ 4` becomes `2 ^ (3 ^ 4)`. This is the integer exponentiation function.

The floating point one is `**`, and it is also right associative:

```
2 ** 3 ** 4 == 2 ** (3 ** 4)
```

For associative operators it doesn't matter

For associative operators, it doesn't matter. Left or right, you still get the same result. For these, it's common to just pick one.

For example, `+` is left associative behind the scenes. It doesn't matter, and you'd get the same result as if it were right associative.

`++`, the concatenation operator is right associative, but again, it doesn't matter.

For backticks

Remember that we can surround a binary function in backticks to turn it into a binary infix operator. For example: `8 `div` 2`

When we do this, Haskell just chooses left associativity arbitrarily.

```
8 `div` 2 `div` 4 == (8 `div` 2) `div` 4
```

Honestly you should probably use parentheses if you are going to do this.

Also remember, function calls always have the highest precedence. Operators can have lower precedence.

\$ vs .

Remember that `.` is the function composition operator. It takes two functions and returns a function that combines them (right first, then left).

`$` is the function application operator. It takes a function and an argument, and then applies the function to the argument. It has a super low precedence, so it is good for avoiding parentheses.

The types are different:

```
(.) :: (b -> c) -> (a -> b) -> a -> c
-- equivalent to (b -> c) -> (a -> b) -> (a -> c) because -> is right associative
-- i.e., take two functions, return a function that takes an a and spits out a c
```

```
($) :: (a -> b) -> a -> b
-- take a function and a value, run the function on the value.
```

Knowledge check 6

1. Consider the xor operator from C. Is it associative?
2. How is associativity different from commutativity? Can you give an example of a function that is associative without being commutative?
3. Write a function that doubles an `Int`. Then write a function that computes double the length of a given string, but use the `$` operator and apply your first function. Don't use any parentheses in the second function. Show its type.
4. Rewrite that function to be point free, and use the `.` operator instead of `$`

Knowledge check 6 answers

1. Yes. $(x \wedge y) \wedge z == x \wedge (y \wedge z)$.
2. Commutativity is when $a R b == b R a$ for some relation R . String concatenation is not commutative: `"hello" ++ "world"` is not `"world" ++ "hello"`. However, as we stated before, it is associative: `"hi" ++ ("hello" ++ "hey") == ("hi" ++ "hello") ++ "hey"`

3 and 4:

```
twoX :: Int -> Int
twoX = (2*)
```

```
twiceLength :: String -> Int
twiceLength s = twoX $ length s
```

```
twiceLength' :: String -> Int
twiceLength' = twoX . length
```

Questions?

Project

The project has you writing some simple recursive code, just like we've seen.

We'll be working with strings, which are just lists of characters.

Let's take a look!

Homework

Work through [Recursion](#), List [mapping](#), [folding](#), [data types](#), and [pattern matching](#).

Most of this material has already been covered. You can skim through as long as the exercises are easy for you.

If they aren't, please read carefully first.

Homework (2)

More codewars! You should now be able to do [7-kyu difficulty Haskell problems](#).

Remember to look at the answer when you've fully submitted. It might show an elegant solution that you never considered.

There might be a prize for doing lots of codewars later in the semester. I strongly recommend doing it!

Questions?

The quiz format

Quizzes will have 4 questions, each worth 25% of the total credit.

Typical kinds of questions:

- What is the type of an expression
- Provide an expression or function definition with the given type
- Write a function that satisfies some requirement
- Write a data type that satisfies some requirement

Practice quiz 1

1. (25%) Suppose we want a function that takes two lists of `Integer` and adds them piecewise. So `addPiece [1,2,3] [4,5,6] == [5,7,9]`. What is the type of this function?
2. (25%) Define that function. Also make it so that the result length is the same as the smaller input length so `addPiece [1,2,3] [4,5] == [5,7]`
3. (25%) Suppose we wanted to *require* that the two lists have the same length, and if they didn't, to return `Nothing`. Now what should the new type be?
4. (25%) Define that function. Make it so that `addPiece [1,2,3] [4,5,6] == Just [5,7,9]` but `addPiece [1,2,3] [4,5] == Nothing`

Practice quiz 1 answers

```
1. addPiece :: [Integer] -> [Integer] -> [Integer]
2. addPiece [] _ = []
   addPiece _ [] = []
   addPiece (x : xs) (y : ys) = (x + y) : (addPiece xs ys)
3. addPiece' :: [Integer] -> [Integer] -> Maybe [Integer]
4. addPiece' [] [] = Just []
   addPiece' x [] = Nothing
   addPiece' [] y = Nothing
   addPiece' (x : xs) (y : ys) =
     case addPiece' xs ys of
       Nothing -> Nothing
       Just t -> Just $ (x + y) : t
```

Practice quiz 2

1. (25%) Define a version of `$` named `ap2` that takes a binary function and *two* arguments instead of 1. It should call the function its given on both arguments.
2. (25%) What is its type?
3. (25%) What is the type of `ap2 (+) (2 :: Int)` ?
4. (25%) What does that function do?

Practice quiz 2 answers

2. `ap2 :: (a -> b -> c) -> a -> b -> c`

1. `ap2 f x y = f x y`

3. `Int -> Int`

4. It adds 2 to things. It is equivalent to `(2+)`.

Practice quiz 3

1. (25%) Suppose we want a function that, given an `Int` `x` and a list of anything, returns the first `x` elements of the list. What should its type be?
2. (25%) Define that function. If the given list is too short, just take its remainder.
3. (25%) Define a function *point free* named `floop` that reverses a string and takes its last `3` elements in order. So `floop "hello" == "o11"`. You will only get credit for point free answers. You can assume that `reverse` is already defined.
4. (25%) What is the type of `floop`?

Practice quiz 3 answers

```
-- 1.  
take' :: Int -> [a] -> [a]  
-- 2.  
take' 0 _ = []  
take' _ [] = []  
take' n (x : xs) = x : take (n - 1) xs  
-- 4.  
floop :: String -> String  
-- 3.  
floop = take' 3 . reverse -- first reverse, then take 3
```

Practice quiz 4

1. (25%) Define a data type that can either hold an integer or a string.
2. (25%) Define a function `myStrLen` which returns the length of the string of that type, but returns `Nothing` if it has an int.
3. (25%) Define a function "convert", that takes your data type. If it has an integer, return the same data type but with the empty string. If it is a string, leave it alone.
4. (25%) Give types to the functions from 2. and 3.

Practice quiz 4 answers

```
data IntOrString = ItsAnInt Int | ItsAString String
```

```
myStrLn :: IntOrString -> Maybe Int
```

```
myStrLn (ItsAnInt _) = Nothing
```

```
myStrLn (ItsAString s) = Just $ length s
```

```
convert :: IntOrString -> IntOrString
```

```
convert (ItsAnInt _) = ItsAString ""
```

```
convert x = x
```

Ask an AI

Ask an AI to generate a practice quiz for you like the above! Give it the slides starting with "# Quiz Format" and up to and including this slide. Or just the whole markdown!

Then, ask it to grade you. I like to use this scale:

1. 0 points off for extremely minor things. Misspellings or missing grouping operators that are clearly intended.
2. 5 points for mistakes that cause the code to fail but are more than just minor mistakes. For example a small type error where it's clear you get the big idea but, e.g., applied the applicative to too many arguments or something.
3. 10 points for bigger mistakes, like type errors that can't work, but there's still "more than half" of the understanding demonstrated.
4. Zero points total if there are several major mistakes or it looks like you're guessing.

Questions?